

Machine Learning for Molecular Engineering
3/7/10/20.01 (U) 3/7/10/20.C51 (G)
Spring 2025

Problem Set #4

Date: April 15, 2025

Due: April 28, 2025 @ 11:59 pm

Instructions

- This problem set has one task with several sub-questions. This problem set doesn't require a GPU. Some are marked grad version, which are required for graduate students (X.C51) but optional for others. Points for all students are in [blue](#), while grad-only points are in [orange](#). The total points are 75 for undergraduates and 100 for graduates.
- To get started, open your Google Colab or Jupyter notebook starting from the problem set template file [pset4.ipynb](#) ([direct Colab link](#)). You will need to use the data files [here](#). Make your own copy of this template to save changes, by selecting "Save a copy in Drive".
- Collaboration is encouraged and AI tools are permitted, but submitting work that is not your own is plagiarism. Any collaboration or assistance from an LLM (including utilities present within Google Colab) should be described at the end of your submission.
- Upon completion, submit your IPython Notebook `pset4.ipynb` to [Gradescope](#). Ensure your submission includes *all* necessary code to be run without error, with all plots and outputs. Comments are encouraged; put answers to conceptual questions in Markdown/Text cells.

Background

SMILES for representing molecules

Whether it's designing specific ligands for enzyme inhibition, predicting the biological activity of small molecules, or exploring chemical space for drug discovery, understanding how to encode chemical structures is essential for computational biology (as well as computational chemistry, of course). The simplified molecular-input line-entry system (SMILES) is a text-based notation describing the structure of molecules using short ASCII strings. In terms of a graph-based computational procedure, SMILES strings are generated by printing the symbol nodes encountered in a [breadth-first traversal](#) of the graphs, typically excluding hydrogen atoms. Any cycles are broken so that the graph becomes an acyclic (tree-structured) graph. Numbers indicate connections between non-adjacent characters in the SMILES string; parentheses are used to indicate points of branching on the tree.

Every molecule can be represented by multiple SMILES strings. For example, `CCO`, `OCC`, `C(O)C`, and `[CH3][CH2][OH]` all specify the structure of an ethanol molecule. There exist algorithms that can reproducibly generate a canonical SMILES string for a molecule. However, an arbitrary string of characters found in SMILES does not necessarily generate a SMILES string or a molecule; there is a fairly complex grammar that must be carefully respected.

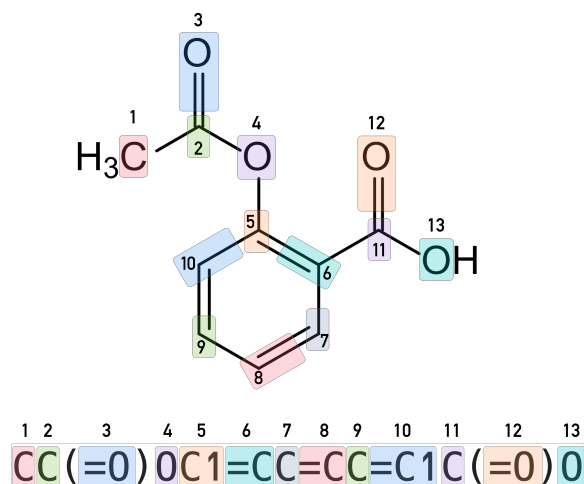


Figure 1: A molecular structure can be represented by SMILES string ([source](#))

In order to convert these SMILES strings into a vector representation, the cheminformatics community has developed a variety of algorithms to numerically encode the structural information of molecules. **Morgan fingerprints**, also known as circular fingerprints, represent the molecular graph as a unique bit string that encodes the molecular topology. You can understand it as a bit-vector categorical encoding with 1 for substructures present in the molecule and 0 for those not present. Some implementations of fingerprints use counts of how many times a substructure is present, instead of just 0s and 1s for absence/presence. Morgan fingerprints are typically high dimensional to maximize their descriptive power and sizes of 512, 1024 or 2048 are common (you can choose the number of bits when generating these fingerprints with the Morgan Algorithm). Morgan fingerprints provide a vector representation of molecules that is nearly unique for each molecule (although fingerprint collisions for non-identical molecules are possible). Note that we provide the code to convert the SMILES strings to 2048-dimensional molecular fingerprints. The fingerprints are computed with the Morgan algorithm implemented in the [RDkit](#) package.

Part 1: Dimensionality Reduction for Molecular Representations

In this part, you will be performing dimensionality reduction analysis on a set of candidate molecules assessed for inhibition of the protein **Janus kinase 2**, also called JAK2. Inhibitors of JAK2 have applications for the treatment of cancers and inflammatory diseases; thus, there is interest in identifying molecules with a high pIC50 for JAK2. Especially relevant in pharmacology and biochemistry, pIC50 refers to the negative logarithm of the IC50 value, where IC50 stands for the half-maximal inhibitory concentration and measures the potency of a substance in inhibiting a specific biochemical function, often the activity of a protein. A higher pIC50 value indicates a higher potency of inhibition.

The dataset is from Popova et al. (2018) [1], which used deep reinforcement learning to generate a dataset of 1,911 small molecules with pIC50 values for JAK2. The goal of this part is to use dimensionality reduction to assess if the representative dimensions of candidate molecules' chemical structures fundamentally organize them by their predicted pIC50 for JAK2.

Part 1.1 (10 points, Grad only) Choosing radius/bits for Morgan fingerprints

You will notice that we set the radius and number of bits to 3 and 2048, respectively, in the code we provided to convert SMILES to Morgan fingerprints. Do some quick researching on the radius and number of bits parameters in RDKit's Morgan fingerprinting.

Task: First, provide a one-sentence description of what the radius represents and another of what the number of bits represents. Then how does adjusting the radius parameter affect the granularity of the motifs captured by the fingerprints, and how does this relate to the choice of the number of bits?

Part 1.2 (15 points) Principal Component Analysis on Molecular Fingerprints

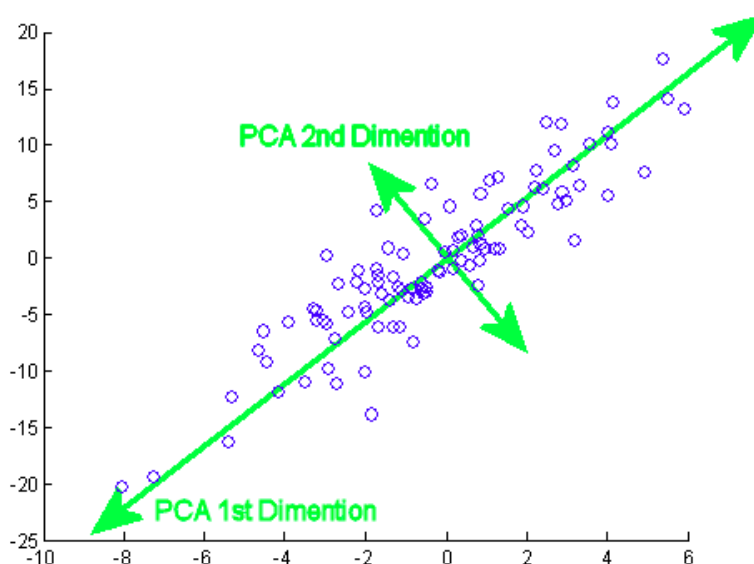


Figure 2: An example PCA analysis on a 2D dataset. The principal components are marked in green arrows. ([source](#))

Principal Component Analysis (PCA) is a dimensionality-reduction method that is often used to tackle high-dimensional, large data sets. In PCA, a large set of variables is transformed into a smaller one that contains as much of the information in the larger set as possible. PCA performs a linear transformation on your data by re-projecting in onto a new orthogonal basis. These new basis is chosen such that the fewest basis vectors best explain the variance of your data. Because of ease of visualization, it is common to just interrogate the first two dimensions (also called eigenvectors or components) of the PCA, but its possible to select additional dimensions.

Task 1: Run the code we provide to load in `jak2.csv`, a dataframe with two columns for SMILES strings and pIC50 values of 1,911 molecules for JAK2. Next, run the provided code to convert these SMILES strings into 2048-bit Morgan fingerprints. Each row in the fingerprint dataframe `jak2_fp` corresponds to the row of the respective molecule.

Task 2: Reduce the 2048-dimensional space of the fingerprint data to a space of just 100 dimensions with PCA. You should use `sklearn.decomposition.PCA()` to generate this embedding.

Now, visualize the first two components of your data in a 2D scatter plot and color each molecule by its pIC50. What is the total percent explained variance of the first 50 principal components? The attributes of `sklearn.decomposition.PCA()` should help with this question. What patterns do you observe (if any)?

Part 1.3 (15 points) t-SNE analysis on Molecular Fingerprints

PCA applies linear transformations on your data, but may struggle to tease out subtle underlying patterns in your data. nonlinear transformations can distort or compress the dimensions much more effectively and can uncover much more complex structure in the dataset. There are two popular choices for nonlinear dimension reduction: UMAP and t-SNE. t-Distributed Stochastic Neighbor Embedding (t-SNE) performs a stochastic nonlinear transformation on your data and can be implemented with `sklearn.manifold.TSNE()`. In general, t-SNE is a random algorithm so you will have different looking embeddings every time; it is also sensitive to perplexities which balance attention between local and global aspects of your data.

Task: Apply t-SNE on the 100-dimensional PCA embeddings you obtained from previous part with `n_components=2` with `perplexity` $\in \{2, 30, 500\}$ separately and produce three labeled t-SNE plots. The t-SNE computation might take a few minutes. Do you notice any differences in the clustering? For the `perplexity=30` plot, do you observe any patterns?

Part 1.4 (35 points) Are the low dimensional embeddings meaningful?

In this question, you're asked to make sense of the extracted low-dimensional embeddings and how they may relate to prediction performance in a supervised learning task. You will train a random forest classifier and then relate the performance of this supervised to the embeddings you learned in the previous part. Let's imagine a biopharmaceutical company is only willing to invest resources into making molecules that have a pIC50 ≥ 9.5 , as they consider this threshold important in discerning potent, effective inhibitors of JAK2.

Task 1: Discretize the data by classifying any molecule with a pIC50 ≥ 9.5 as effective (i.e., 1) and any with a pIC50 < 9.5 as ineffective (i.e., 0). Capture this information as an additional column called `is_effective` in the `jak2` dataframe.

Task 2: Next, split your data into 10 folds using `sklearn.model_selection.KFold()` and train a random forest classifier with 100 trees, i.e. `n_estimators=100`. Use the full fingerprint (not the low-dimensional embedding) as your featurization for the random forest. Just as you did in the first problem set, use `sklearn.ensemble.RandomForestClassifier()`. Test each fold with models trained on the other 9 folds, and record your predictions in the `jak2` dataframe or an array.

Task 3: Classify your predictions in 4 categories: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). Make a scatter plot with the `perplexity=30` t-SNE embeddings, and color each of the four categories above with different colors. What pattern(s) do you observe (if any)?

Part 1.5 (10 points) UMAP analysis on Molecular Fingerprints

As described above UMAP is another alternative to PCA that can better capture nonlinearities. This is implemented with the `umap` library in Python as `umap.UMAP()`. Read the [documentation](#) to

understand some of the keyword arguments.

Task: Apply UMAP, just as you did with t-SNE, on your 100-dimensional PCA embeddings you obtained earlier. Use `n_components=2`, `n_neighbors=10`, and `min_dist=1.0` and produce a plot with effective and ineffective molecules (as defined above) in different colors.

Part 1.6 (15 points, Grad only) Visualize structures in UMAP clusters

One approach that may help us interpret our latent space is clustering. To make this a bit easier, we will only explore the effective compounds and look at the common features with each cluster. DBSCAN is one effective algorithm for clustering such data.

Task: Use `sklearn.cluster.DBSCAN` to cluster the effective molecules with parameters `eps=0.6` and `min_samples=4` to draw insights on commonalities between groups of these molecules. Visualize, **only** the effective clusters, by plotting the UMAP results with unique colors and labels. Now pick a particular cluster and show the member structures using the provided skeleton code. Comment on the similarity of structures.

References

- [1] Popova, M., Isayev, O. & Tropsha, A. Deep reinforcement learning for de novo drug design. *Science Advances* **4** (2018).